

LESSON 03 / STAGE ONE

Enumeration & Its Optimization

枚举算法与优化

暴力枚举 · 数学性质减层 · P7071 优秀的拆分精讲

把每一种可能都老老实实试一遍，再用数学性质把试不完的部分一刀砍掉——枚举，是"暴力"与"聪明"的合奏：范围卡得准，检验写得全，答案自然浮出水面。

03

授课班级

中预 2026 级 B 班

主讲

信息教研组

时间

2026 年 10 月

共 10 道课堂实践题 · 含洛谷题号

1

会列"所有可能"——枚举三要素

先想清楚三件事：枚举谁（对象）、从几到几（范围）、怎么验货（检验）。三要素一旦确定，代码骨架就定了——这是本课第一关键点。

2

会用数学性质砍枚举层

奇偶性先砍一半，整除性再砍一刀：P7071 里"奇数必无解"一条判断，一半数据瞬间出答案——本课题里"优化"二字的主角。

3

会用空间换时间——打表、前缀和、桶

把"第二层枚举"变成"查一次表"：A-B 数对查桶、火柴棒打表、刺痛值滑窗，10 万级数据也能瞬间出答案。

4

有复杂度意识——动手前先算账

循环套几层 × 每层多少次？估出来超过 10^8 就要换打法。"暴力"不是无脑，而是"算得清账的暴力"。

课堂安排

建议节奏，可按学情调整

15 分钟

概念讲解

25 分钟

例题精讲 P7071

55 分钟

上机实践（10 题）

10 分钟

总结答疑

承上启下

第 2 课我们学会了"照题意一步步做"；这一课把问题反过来——不告诉你怎么走，让你把所有走法都试一遍，再聪明地少试。

一句话定义

枚举，就是把问题所有可能的情况一个不落地列出来，逐个检验，把满足条件的留下来——朴素、完整、不会漏。

1 对象要选对

枚举"准"大有讲究。P1008 里枚举第一个三位数只要 207 个候选；老老实实枚举 9 个数字的全排列就是 36 万次——对象选得好，快上千倍。

2 范围要卡紧

范围每砍一半，时间就省一半。整除、奇偶、大小关系都是天然的"范围剪刀"——先用数学性质筛一遍，再动手写循环。

3 检验要写对

枚举出候选只是第一步，还要用 if 逐个验货。检验写漏一种情况（比如忘了 0 不能出现），整道题就从"对"变成"错"。

生活里的一次"枚举" —— 试密码箱（今天核心题的原型！）

拿到密码箱

000 ~ 999，一共 1000 种可能



逐个试

000、001、002……一个不跳



每次验货

转一下锁：开了没有？



得到答案

开了就是密码；全没开则不在范围

暴力，但保证不漏——这就是枚举的底气。剩下的功夫，全在"怎么少试几把"。

C++ 视角

一至多重循环 + if 检验，就是枚举的全部骨架。CSP-J 入门级 T1 常客：数据范围往往刚好"逼你"先算复杂度——把范围卡准、检验写全，就是最稳的得分手段。

关键点一：枚举三要素——对象、范围、检验

三要素 checklist：写枚举前先问自己三个问题

- 1

对象：枚举谁？

找"定一个、牵全部"的量。三连击里定下第一个数 x ， $y = 2x$ 、 $z = 3x$ 全跟着定了——枚举 1 个变量，而不是 9 个数字的全排列（36 万种）。
- 2

范围：从几到几？

范围是约束"推"出来的：1~9 不重不漏 $\rightarrow x$ 最小 123； $z = 3x$ 必须是三位数 $\rightarrow x \leq 987 \div 3 = 329$ 。两刀砍完只剩 207 个候选。
- 3

检验：怎么算对？

cnt[10] 桶逐位计数： x 、 y 、 z 的 9 个数字，1~9 各恰好出现一次、0 不出现才算过关。检验是枚举的"验货环节"，一种情况都不能漏。

一句话记住

对象、范围、检验三票齐了再动手——三要素里有两票（对象、范围）都花在"少枚举"上，优化的功夫在写代码之前。

主循环伪代码：三要素各就各位

```
for x = 123 .. 329:           // ② 范围 (① 对象是 x)
    y = 2 * x;  z = 3 * x
    cnt[0..9] 全部清零
    把 x、y、z 的 9 个数字
    逐位丢进 cnt[] 计数
    ok = (1..9 各恰好一次)
        且 cnt[0] == 0
    if ok: 输出 x y z         // ③ 检验
```

三个约束（三位数、1 : 2 : 3、数字 1~9 不重不漏）用掉两个来卡范围，只剩一个留给 if——这是枚举优化的常用套路。

常见错位

范围拍脑袋开成 100~999：900 组候选大半是废票，还可能把数字 0 混进 cnt 里忘了排除。范围不是试出来的，是用约束"推"出来的。

关键点二：用数学性质砍枚举层

一道真题：把 n 拆成互不相同的 2 的正整数次幂之和

6 = 4 + 2 是优秀的拆分；8 = 4 + 4 不行（4 重复）；10 = 7 + 3 也不行（7、3 不是 2 的幂）。n ≤ 10⁷ 时 2²⁴ 已超过它——暴力做法：24 个幂每个"选或不选"，2²⁴ ≈ 1700 万种子集，1 秒时限擦线赌博。

性质① 奇数必无解

2、4、8……全是偶数，偶数之和还是偶数。n 是奇数 → 直接输出 -1：一半的数据瞬间判掉，循环都省了。

性质② 相同幂可合并

2^k + 2^k = 2^{k+1}，所以最优拆法里每个幂最多出现一次 → 从大到小贪心"能拿就拿"，n = 10⁷ 也只跑 24 轮。

还要砍第三刀：贪心为什么对？从大到小"能拿就拿"，剩下的零头刚好被小幂补齐——选出来的组合恰好就是 n 的二进制表示，天然互不相同，方案唯一。

本课反复出现的思维动作

先读懂数据的"形状"（奇偶、整除、单调），再决定循环怎么写——性质就是剪刀。

主循环伪代码：两条性质各占一行

```
读入 n

if n % 2 == 1:                // 性质① 奇数无解
    输出 -1; 结束

for i = 24 down to 1:        // 2^24 > 10^7
    if n ≥ 2^i:
        选下 2^i;  n -= 2^i

输出选下的所有幂

(顺序自然从大到小)
```

注意输出顺序：从大到小枚举，选下的幂天然从大到小排列——与题目要求的输出顺序一致，连排序都省了。

效果对照

O(2²⁴) ≈ 1.7×10⁷ 子集枚举 → O(24) 贪心：一条数学性质砍掉约 70 万倍的枚举量——这就是本标题里"优化"二字的分量。

关键点三：打表、前缀和、桶——空间换时间

三件"省力气"的武器

A 预处理打表（T05 火柴棒）

先扫一遍算出 $need[i] = \text{数 } i \text{ 需要几根火柴}$ ，枚举 $a、b$ 时查表 $O(1)$ 。不做预处理，每个候选都要重新数一遍位。

B 桶计数（T04 A-B 数对）

$A - B = C$ 变形为 $B = A - C$ 。每个数出现几次先数进桶，枚举 A 时"查一次桶"代替"再扫一遍数组"—— $O(n^2)$ 变 $O(n)$ 。

C 前缀和 / 滑动窗口（T03 刺痛值）

连续 m 个数的最小和：窗口右进左出只做一次加减，不用每个窗口从头加—— $O(n \cdot m)$ 变 $O(n)$ 。

共同的本质

枚举优化的三板斧，本质都是"把重复算过的东西存起来"：数过的位数存成表、数过的个数存进桶、加过的和留在窗口里——用一点空间，换大量时间。

复杂度对照账（本课三道代表题）

题目	朴素枚举	优化后
P1149 火柴棒等式	每个候选重新数位	打表后 1.2×10^6 次查表
P1102 A-B 数对	双层 for 4×10^{10} 必炸	查桶 $O(n) = 2 \times 10^5$
P1614 爱与愁的心痛	每窗从头加 6×10^{10}	滑动窗 $O(n) = 2 \times 10^5$

算账标准

朴素做法一旦超过 10^8 次基本运算，1 秒时限基本告吹；优化后 10^6 量级"闭眼过"。动手前先列这张表，再决定写不写优化。

常见误区

不是所有题都要优化： $n \leq 1000$ 的双重循环才 10^6 次，朴素枚举完全够用——先算账再动手，别为了优化而优化。

本课十题总览

编号	题目	来源	难度	枚举对象与优化手法
T01	优秀的拆分 P7071	CSP-J 2020 T1	★★★	从大到小枚举 2 的幂：奇偶性砍半，24 轮贪心
T02	三连击 P1008	NOIP1998 普及组	★★	枚举第一个数 $x \in [123, 329]$ ：约束反推范围
T03	爱与愁的心痛 P1614	洛谷经典	★★	枚举窗口：滑窗右进左出， $O(n \cdot m)$ 变 $O(n)$
T04	A-B 数对 P1102	洛谷经典	★★★	枚举 A 查桶： $B = A - C$ ，查表代替第二层
T05	火柴棒等式 P1149	NOIP2008 提高组	★★★	枚举 a、b：need[] 打表，数位不重算
T06	选数 P1036	NOIP2002 普及组	★★★	组合枚举（选 / 不选）：试除到 $\sqrt{\text{sum}}$ 判素
T07	烤鸡 P2089	洛谷经典	★★	十种配料全枚举：可行性剪枝砍路径
T08	扫雷游戏 P2670	NOIP2015 普及组	★★	枚举格子 $\times 8$ 方向：方向数组免写 8 个 if
T09	统计天数 P1567	洛谷经典	★	线性扫描：一段一段数，一遍跑完
T10	欢乐的跳 P1152	洛谷经典	★★	枚举相邻差：桶标记验「1..n-1 齐全」

全部 10 题的参考代码均已通过洛谷官方样例验证。课堂节奏：T01 精讲 5 页；T02 – T10 每题 4 页（题面 → 思路 → 方法 → 代码），可按学情取舍。难度为课堂参考值，均不超过 CSP-J T1 水平。

题面描述

一般来说，一个正整数可以拆分成若干个正整数的和。我们称一种拆分是「优秀的」，当且仅当 n 被分解成了若干个互不相同的 2 的正整数次幂之和。

例如： $6 = 4 + 2$ 是优秀的拆分；

反例： $8 = 4 + 4$ 不行（ 4 重复了）； $10 = 7 + 3$ 也不行（ 7 、 3 不是 2 的幂）。

给定正整数 n ，判断它是否存在优秀的拆分；若存在，给出具体的拆分方案。

2 的正整数次幂： 2 、 4 、 8 、 16 ……（ $2^0 = 1$ 不算—— 0 不是正整数！）

题目保证：方案存在时，拆分方案是唯一的。

思路预告

别急着写循环——先把「 2 的正整数次幂」这个限制翻译成数学性质，答案会自己跳出来。第 5 页已经预告了两条性质，这里正式派上用场。

输入格式

输入只有一行，一个整数 n ，代表需要拆分的数。

输出格式

无解输出 -1 ；有解则从大到小输出拆分中的每个数，相邻两数用空格隔开。

数据范围

$1 \leq n \leq 10^7$ （ $2^{24} = 16777216$ 已超过它）。

三组官方样例

输入	输出
6	4 2
7	-1
126	64 32 16 8 4 2
样例 3 的一行里藏着 6 个互不相同的 2 的幂——把 126 的二进制写出来看看？	

题面 → 代码：暴力为何险，贪心为何稳

主循环伪代码：一条性质一行

```
读入 n

if n % 2 == 1:  输出 -1; 结束      // 性质① 奇数无解

for i = 24 down to 1:              // 2^24 > 10^7
    if n ≥ 2^i:
        选下 2^i;  n -= 2^i      // 能拿就拿

输出选下的所有幂（自然从大到小）
```

暴力子集枚举要 $2^{24} \approx 1.7 \times 10^7$ 次判断——能过但纯属赌博；贪心只跑 24 轮，稳稳收工。

贪心选出的幂集合恰好就是 n 的二进制表示里那些 1 对应的位权——由于 n 是偶数，二进制末位必为 0，所以不会出现 2^0 ，正整数次幂的约束自动满足。

Q1 · 为什么从大到小？

每个幂最多选一次（性质②），大幂先占坑，剩下的零头刚好被小幂补齐——这正是 n 的二进制表示，方案唯一。

Q2 · 为什么奇数直接 -1？

2、4、8……全是偶数，偶数怎么加都是偶数；奇数无论怎么拆都无解。一半数据瞬间判掉。

Q3 · 为什么不会剩下 1？

n 是偶数才进循环；从大到小拆到最后剩下的也是偶数，最小剩 2—— 2^0 从头到尾没机会出现。

一句话

奇数砍一半、互不相同砍分支——两条性质一上，24 轮循环稳稳收工，这就是「枚举 + 优化」的完整闭环。

状态设计 + 样例手推：126 → 64 32 16 8 4 2

1

n：还没拆完的剩余部分——每选一个幂就减一笔。

第 3 课：枚举算法与优化

课堂例 T01 · 优秀的拆分 (P7071)

2

ans 序列：选下的幂，按从大到小输出（枚举顺序天然有序）。

状态清单：程序里要维护什么

3

循环变量 i：当前尝试的幂次，从 24 到 1。

轮次 i	2 ⁱ	n (判断前)	n ≥ 2 ⁱ ?	动作 / 剩余 n
24	16777216	126	否	跳过 / 126
6	64	126	是	选 64 / 剩 62
5	32	62	是	选 32 / 剩 30
4	16	30	是	选 16 / 剩 14
3	8	14	是	选 8 / 剩 6
2	4	6	是	选 4 / 剩 2
1	2	2	是	选 2 / 剩 0 ✓

两条对照线

① 输出 64 32 16 8 4 2 = 官方样例 3，从大到小一行搞定；② 若 n = 7：第一行性质①就直接输出 -1，连循环都不进。第 1、2 组样例由此全覆盖。

手推 = 人肉当 CPU

左列写状态，右表写动作，一行一行走——写代码前先在纸上通一遍样例。

参考代码：24 轮贪心，一气呵成

```
#include <cstdio>
int main() {
    long long n;
    scanf("%lld", &n);
    if (n % 2 == 1) {          // 性质① 奇数必无解
        printf("-1
");
        return 0;
    }
    bool first = true;        // 控制空格输出
    for (int i = 24; i >= 1; i--) {    // 2^24 > 10^7
        long long p = 1LL << i;
        if (n >= p) {          // 能拿就拿：贪心
            n -= p;
            if (!first) printf(" ");
            printf("%lld", p);
            first = false;
        }
    }
    printf("
");
    return 0;
}
```

结构一览

没有 struct、没有数组——一个 n、一层 for、一个 if，全题完事。唯一要记住的是 $2^{24} > 10^7$ ，循环上界取 24。

三步对号入座

if (n % 2) 行 = 性质①奇偶判断；for 行 = 性质②从大到小；if (n ≥ p) 行 = 「能拿就拿」贪心。与第 9 页伪代码一一对应。

两个好习惯

① 2 的幂用 1LL 左移 i 位写，long long 不越界；② 用 first 标记控制空格，行尾不多打空格——很多 OJ 会判 Presentation Error。

提交提示

课后亲手敲一遍并提交洛谷 P7071。三组官方样例 (6→4 2; 7→-1; 126→64 32 16 8 4 2) 及边界值 1、2、 10^7 均已通过验证脚本验证。

错1 忘判奇数，硬拆到底

$n = 7$ 若硬跑循环，最后会剩 1 拆不掉——2 的正整数次幂里没有 1。输出残缺或错判有解，先判奇数一步到位。

错2 把 $2^0 = 1$ 也当成拆分项

$1 = 2^0$ ，但 0 不是正整数！循环从 $i = 24$ 到 1，天然避开 2^0 ；如果自己写 `while(n > 0)` 一路减，就会踩坑。

错3 2 的幂用 int 硬算

1 左移 24 位装得下 int，但顺手写成 2^i 逐次相乘或取错位极易溢出；统一用 1LL 左移写幂，n 又是 long long，一步到位。

复杂度小账

$O(24)$

暴力子集枚举 $2^{24} \approx 1.7 \times 10^7$ ，1 秒时限属于「擦线赌博」——机器快一点慢一点全看命。贪心只跑 24 轮， $n = 10^7$ 也是瞬间完成。这就是数学性质的分量：把「碰运气」变成「必然」。

本讲要点回收

奇偶判 → 从大到小 → 能拿就拿。十二个字，就是这道 CSP-J T1 的全部。

题面描述

将 1 ~ 9 共 9 个数字分成 3 组，分别组成 3 个三位数，且使这 3 个三位数构成 1 : 2 : 3 的比例。试求出所有满足条件的 3 个三位数。

本题没有输入——答案固定，直接把所有解按每行第一个数升序输出即可。

三个约束：① 每个数都是三位数；② 三数之比 1 : 2 : 3；③ 数字 1~9 在三个数中不重不漏地各出现一次。

反例：100 200 300（有 0，且数字重复）；123 246 369（数字重复）。

输入格式

无输入。

输出格式

若干行，每行 3 个数，按每行第 1 个数升序排列。

数据范围

无（本题直接输出全部答案）。

官方样例

输入	输出
无	192 384 576 219 438 657 273 546 819 327 654 981

共 4 行，全部答案——课后可自行核对。

思路预告

枚举对象选谁，效率天差地别：定下第一个数，另外两个自动确定——这个"牵一发动全身"的量就是最佳枚举对象（第 4 页的关键点一）。

思路梳理：枚举一个，牵动三个

三要素逐项落实

对象：枚举谁？

x 定了, $y = 2x$ 、 $z = 3x$ 全跟着定了——枚举 1 个变量, 而不是 9 个数字的全排列 (36 万种)。

范围：从几到几？

$z = 3x$ 必须是三位数 $\rightarrow x \leq 987 \div 3 = 329$; $x \geq 123$ (1~9 不重不漏的最小排布)。207 个候选, 快上千倍。

检验：查什么？

x 、 y 、 z 共 9 个数字逐位计数: 1~9 各恰好 1 次、0 一次都不出现。cnt[10] 桶数组一把搞定。

主循环伪代码

```
for x = 123 .. 329:           // ② 范围
    y = 2x;  z = 3x
    cnt[0..9] 清零
    拆出 x、y、z 的每个数字
    数字 d:  cnt[d]++
    ok = (cnt[0] == 0) 且
           1..9 各恰好 1 次
    if ok: 输出 x  y  z      // ③ 检验
```

一句话记住

三个约束用掉两个卡范围, 只剩一个留给 if——"范围是推出来的, 不是拍出来的", 第 4 页的原则在这里落地。

方法拆解 + 样例手推：x = 192 走一遍

方法三步

① 定对象

$x = 192$: $y = 2x = 384$ 、 $z = 3x = 576$ ，三位数与 $1 : 2 : 3$ 自动满足。

② 拆数字

1 9 2 3 8 4 5 7 6 逐位丢进 cnt——9 个格子各记一笔。

③ 查桶

$\text{cnt}[0] = 0$ ✓; 1~9 各恰好 1 次 ✓ → 输出 192 384 576。

顺序白送

答案恰好按 x 升序——枚举顺序天然有序，连排序都省了。

步骤	操作	结果
定 $x = 192$	算 $y = 384$ 、 $z = 576$	1:2:3 ✓ 三位数 ✓
拆数字计数	1,9,2,3,8,4,5,7,6 逐位 ++	cnt 共 9 笔
查 0	$\text{cnt}[0] == 0$	✓
查 1..9	各恰好 1 次	✓ 输出

全部 4 组解

192 384 576、219 438 657、273 546 819、327 654 981——已用验证脚本对过官方输出，与本页手推同构。

```
#include <cstdio>
int main() {
    for (int x = 123; x <= 329; x++) {    // 对象+范围
        int y = 2 * x, z = 3 * x;
        int cnt[10] = {0};                // 循环内定义,自动清零
        int t[3] = {x, y, z};
        for (int i = 0; i < 3; i++)
            for (int v = t[i]; v > 0; v /= 10)
                cnt[v % 10]++;            // 逐位计数
        bool ok = (cnt[0] == 0);
        for (int d = 1; d <= 9; d++)
            if (cnt[d] != 1) ok = false;    // 1~9 各恰好一次
        if (ok) printf("%d %d %d\n", x, y, z);
    }
    return 0;
}
```

结构一览

双层 for (外枚举 x、内拆数字) + cnt 桶 + 一轮检查——纯数组无 STL，初学者友好。cnt 定义在循环内，每轮自动清零。

三步对号入座

for x 行 = 对象 + 范围；内层两行 = 把候选"摊开"计数；if (ok) = 检验输出。与第 14 页伪代码一一对应。

两个好习惯

① cnt 定义在循环体内自动清零，不会让上一轮的脏数据串场；② 输出行末直接换行，格式干净。

提交提示

课后亲手敲一遍并提交洛谷 P1008。输出共 4 行，已通过验证脚本对拍官方答案。

T03 | 爱与愁的心痛（P1614）

题面描述

最近有 n 件不爽的事，每件有一个刺痛值。爱与愁大神想知道：连续 m 件刺痛值的和最小是多少？

「连续」是关键词——只能取相邻的一段，不能挑挑拣拣。

本课定位：滑动窗口的入门题——"前缀和 / 滑窗"这把武器的第一次实战。

思路预告

暴力每个窗口从头加要 $O(n \cdot m) \approx 6 \times 10^{10}$ ，必炸——相邻窗口共享 $m-1$ 个数，让窗口"滑"起来。

输入格式

第一行两个整数 $n\ m$ ；接下来 n 行，每行一个整数 a_i ，代表第 i 件事的刺痛值。

输出格式

一行一个整数：连续 m 件刺痛值的和的最小值。

数据范围

$1 \leq m \leq n \leq 2 \times 10^5$ 。

官方样例

输入	输出
8 3 1 4 7 3 1 2 4	6
答案是 $3 + 1 + 2 = 6$ ——连续第 4~6 件事。	
3	

从暴力到滑窗

暴力差在哪？

枚举每个左端点，从 $a[i]$ 加到 $a[i+m-1]$ ——相邻两个窗口重复加了 $m-1$ 个数。重复劳动就是优化空间。

滑窗怎么做？

先加好前 m 个数得 s ； i 从 m 到 $n-1$ ： $s += a[i] - a[i-m]$ ，右进一个、左出一个，每步 $O(1)$ ，总共 $O(n)$ 。

为什么是对的？

每个窗口恰好被算一次，共享部分进出抵消——数学上就是"前缀和之差"，第 9 课递推还会再见到它。

主循环伪代码

```
读入 n, m 和数组 a

s = a[0]+...+a[m-1]    // 第一扇窗

best = s

for i = m .. n-1:

    s += a[i] - a[i-m]    // 右进左出

    best = min(best, s)

输出 best
```

复杂度小账

$O(n \cdot m) \approx 6 \times 10^{10} \rightarrow O(n) = 2 \times 10^5$ ：窗口和最大可达 10^8 量级——int 擦线，long long 稳妥；先算账再选类型，老规矩。

方法拆解 + 样例手推：答案是 6

滑窗三步

① 第一扇窗

$a[0..2] = 1 + 4 + 7 = 12$, $best = 12$ 。

② 滑动 5 步

每步一加一减： $12 \rightarrow 14 \rightarrow 11 \rightarrow 6 \star \rightarrow 7 \rightarrow 9$, $best$ 一路刷新。

③ 输出

$best = 6 =$ 官方样例答案 $\checkmark$ 。

重复利用

窗口共 $n-m+1$ 个；"重复利用已经算过的和"是枚举优化的第一直觉。

窗口（连续 3 件）	进 / 出	窗和	best
[1, 4, 7]	初始化	12	12
[4, 7, 3]	+3 -1	14	12
[7, 3, 1]	+1 -4	11	11
[3, 1, 2]	+2 -7	6 $\star$	6
[1, 2, 4]	+4 -3	7	6
[2, 4, 3]	+3 -1	9	6

手推结论

6 步滑完， $best = 6$ ，与官方样例一致——手推一遍，滑窗的正确性肉眼可见。

参考代码：右进左出

```
#include <iostream>
#include <algorithm>
using namespace std;
int a[200005];
int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> a[i];
    long long sum = 0;
    for (int i = 0; i < m; i++) sum += a[i]; // 第一扇窗
    long long best = sum;
    for (int i = m; i < n; i++) {
        sum += a[i] - a[i - m]; // 右进左出 O(1)
        best = min(best, sum);
    }
    cout << best << endl;
    return 0;
}
```

结构一览

一个数组、两个 long long (sum、best)、两层循环——没有套路的"套路题"，重点全在"右进左出"这一行。

三步对号入座

第一个 for = 初始化第一扇窗；第二个 for = 滑动；min = 验货。与第 18 页伪代码逐行对应。

两个好习惯

① 窗口和用 long long 防溢出；② 20 万级数据可加 ios::sync_with_stdio(false) 加速读入。

提交提示

课后亲手敲一遍并提交洛谷 P1614。官方样例 8 3 → 6 已通过验证脚本验证。

题面描述

给出一串正整数数列和一个正整数 C，要计算出所有满足 $A - B = C$ 的数对的个数。

注意：不同位置的数字一样的数对，算不同的数对——1 1 里两个 1 互相配，位置不同就算两对。

本课定位：桶计数的入门题——"查表代替第二层枚举"的完整示范。

思路预告

双层 for 枚举所有数对是 $O(n^2) = 4 \times 10^{10}$ ，必炸——把等式变形，第二层就能"查表"。

输入格式

共两行：第一行两个正整数 n c；第二行 n 个正整数 a_i 。

输出格式

一行，满足 $A - B = C$ 的数对的个数。

数据范围

$1 \leq n \leq 2 \times 10^5$ ；数值可达 10^9 （变量用 long long，桶用 map）。

官方样例

输入	输出
4 1 1 1 2 3	3

三对：(2,1)、(2,1)'（两个 1 各配一次）、(3,2)。

两步走

第一步：变形

$A - B = C$ 等价于 $B = A - C$ 。枚举每个 A ，答案加上「 $A - C$ 出现了几次」——配对变成了查询。

第二步：查表

每个数出现几次，先一次性数进 map 桶 cnt；查询一次 $O(\log n)$ 。总复杂度 $O(n \log n) \approx 3 \times 10^6$ 。

第三步：防坑

C 是正数 $\rightarrow A \neq B$ ，不会自己配自己；"位置不同算不同对" $\rightarrow$ 桶里存「次数」而不是「有没有」，累加自然算上重复。

主循环伪代码

```
读入 n, c
for i = 1 .. n:
    读入 a[i]
    cnt[a[i]]++           // 数进桶
ans = 0
for i = 1 .. n:
    ans += cnt[a[i]-c] // 枚举A查B
输出 ans
```

一句话记住

正难则反：与其枚举"谁配谁"，不如枚举一个、查另一个——变形公式把主动权交回给你。

方法拆解 + 样例手推：答案是 3

手推两步

① 数桶

数列 1 1 2 3, $C = 1 \rightarrow \text{cnt}$: 1 $\rightarrow$ 2 次、2 $\rightarrow$ 1 次、3 $\rightarrow$ 1 次。

② 查桶

A=1: 查 B=0 $\rightarrow$ 0 次; A=2: 查 B=1 $\rightarrow$ 2 次★; A=3: 查 B=2 $\rightarrow$ 1 次★。

③ 合计

$0 + 0 + 2 + 1 = 3$ ✓ 官方样例答案。

次数即答案

同一个数出现几次，查询时就贡献几次——桶存「次数」的设计在这里见效。

枚举 A	查 B = A-C	桶值	累计
1	0	0	0
1	0	0	0
2	1	2 ★	2
3	2	1 ★	3

手推结论

答案是 3，与官方样例一致。排序 + 二分也能做——但桶更直观，第 20 课优先队列还会跟它打交道。

参考代码：map 当桶

```
#include <iostream>
#include <map>
using namespace std;
map<long long, long long> cnt; // 数值可达1e9
long long a[200005];
int main() {
    int n; long long c;
    cin >> n >> c;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        cnt[a[i]]++;           // 先数进桶
    }
    long long ans = 0;
    for (int i = 0; i < n; i++)
        ans += cnt[a[i] - c];  // 枚举A查B
    cout << ans << endl;
    return 0;
}
```

结构一览

map 容器当桶（键和值都开 long long）：数值可达 10^9 ，普通数组开不下下标——数据范围决定容器，这是本页最重要的一句。

三步对号入座

第一个 for = 读入 + 数桶；第二个 for = 枚举 A 查 B。两行核心代码，与第 22 页伪代码一致。

两个好习惯

① A、C、桶全部 long long；② 先数完桶再查询，顺序不能反——边数边查会漏配对。

提交提示

课后亲手敲一遍并提交洛谷 P1102。官方样例 4 1 → 3 已通过验证脚本验证。

T05 | 火柴棒等式 (P1149)

第 3 课 · 枚举算法与优化

NOIP2008 提高组 · 难度 ★★★

题面描述

给你 n 根火柴棍，可以拼出多少个形如 $A + B = C$ 的等式？

规则：加号、等号各占 2 根； A 、 B 、 C 是非负整数（0 可以，但非零数的最高位不能为 0）； n 根火柴必须全部用上； $A + B$ 与 $B + A$ 算两个不同的等式。

本课定位：预处理打表的入门题——“数过的东西存成表”第一次完整落地。

思路预告

每个候选等式都要数 A 、 B 、 C 三个数的火柴数——先打表存起来，别每次重新数。

输入格式

一个整数 n 。

输出格式

一个整数：能拼出的等式数目。

数据范围

$n \leq 24$ （由此可反推枚举上界，见第 26 页）。

官方样例

输入

14

18

输出

2

9

$n = 14$ 时： $0+1=1$ 和 $1+0=1$ ，各用 $6+2+2+4 = 14$ 根。

思路梳理：打表 + 枚举

第 3 课 · 枚举算法与优化

实践题 T05 · 火柴棒等式 (P1149)

三步走

① 先打表

$need[i]$ = 数 i 需要的火柴数。递推： $need[i] = need[i/10] + seg[i\%10]$
(数位分离)，一遍扫到 2222。

② 再枚举

a 、 b 都从 0 到 1111： $need[a] + need[b] + need[a+b] + 4 == n$ 就计数。
+4 是加号和等号。

③ 上界的来历

每位数字最少 2 根 (数字 1)。20 根火柴 (去掉符号)
撑不起更大的数—— $n \leq 24$ 反推出 a 、 $b \leq 1111$ ，又是"约束推范围"。

数字火柴对照表

数字	0	1	2	3	4	5	6	7	8	9
根数	6	2	5	5	4	5	6	3	7	6

复杂度小账

$1111^2 \approx 120$ 万组候选 × 查 3 次表——1 秒内轻松跑完；
不打表则每组候选都要重新数位，白费几倍功夫。

方法拆解 + 样例手推：n = 14 → 2

手推三步

① 换算

n = 14，去掉符号 4 根 → 三个数的火柴和必须是 10。

② 逐对验

0+1=1: 6+2+2 = 10 ✓; 1+0=1 同样 ✓ (题面规定算两个)。

③ 收尾

0+2=2: 6+5+5 = 16 ≠ 10 ✗……全扫完，答案 2 ✓。

打表三步曲

定义 → 扫一遍填表 → 查表。need[0] = 6 别忘了单独赋值——0 进不了递推，单独处理最稳。

等式	need(a)	need(b)	need(c)	+符号	合计	n=14?
0+1=1	6	2	2	4	14	✓
1+0=1	2	6	2	4	14	✓
0+2=2	6	5	5	4	20	✗
2+0=2	5	6	5	4	20	✗

手推结论

官方样例 n = 18 的答案是 9——验证脚本实测一致，课后可以自己把 9 个等式全列出来。

参考代码：预处理 + 双重枚举

```
#include <stdio>
int seg[10] = {6, 2, 5, 5, 4, 5, 6, 3, 7, 6};
int need[2225];          // need[i]: 数i要几根
int main() {
    int n;
    scanf("%d", &n);
    need[0] = 6;
    for (int i = 1; i <= 2222; i++)
        need[i] = need[i/10] + seg[i%10]; // 数位递推
    int ans = 0;
    for (int a = 0; a <= 1111; a++)
        for (int b = 0; b <= 1111; b++)
            if (need[a] + need[b] + need[a+b] + 4 == n)
                ans++;          // +4: 加号、等号各2根
    printf("%d", ans);
    return 0;
}
```

结构一览

seg[10] 一张字符表、need[2225] 一张打表、双重循环一枚举——典型"预处理 + 枚举"结构，三板斧第一斧的完整示范。

三步对号入座

need 递推 = 打表；a、b 双重循环 = 枚举；if (···+4 == n) = 检验。第 26 页的账在这里逐行落地。

两个好习惯

① need[0] 单独赋 6；② 上界 1111 与 2222 都来自 $n \leq 24$ 的推算——写注释说明来源，防止下次看不懂。

提交提示

课后亲手敲一遍并提交洛谷 P1149。官方样例 14→2、18→9 均已通过验证脚本验证。

题面描述

已知 n 个整数 $x_1, x_2, \dots, x_n$ ，以及整数 k ($1 < k \leq n$)。从 n 个整数中任选 k 个整数相加，可得到一系列的和。求和为素数的组合共有多少种。

例如：3 7 12 19 里选 3 个，只有 $3 + 7 + 19 = 29$ 是素数——答案是 1。

本课定位：组合枚举入门——每个数只有「选 / 不选」两种命运，一棵二叉树走完。递归模板今天先混个脸熟，第 21 课 DFS 系统学。

思路预告

「任选 k 个」就是组合枚举——按数组下标顺序"选 / 不选"，天然不重不漏；判素用试除到根号。

输入格式

第一行两个整数 n, k ；第二行 n 个整数 x_i 。

输出格式

一行一个整数：和为素数的组合数。

数据范围

$1 < n \leq 20$ ； $x_i \leq 5 \times 10^6$ （和最大 10^8 ，判素试除到根号即可）。

官方样例

输入	输出
4 3 3 7 12 19	1
四选三共 4 种组合，恰好 1 种和为素数。	

三个要点

怎么不重不漏地枚举组合？

按「到第 pos 个为止」推进：每个数都有选 / 不选两条岔路；下标只往右走 → 每个组合恰好枚举一次，天然去重。

边界怎么写？

chosen == k 时验货：isPrime(sum) 则 ans++；pos == n 时回头。两条边界一夹，二叉树严丝合缝。

判素怎么不超时？

试除到 $i \times i \leq \text{sum}$ ：和最大 10^8 ，根号和 = 10^4 ；大多数偶数一碰 2 就返回，1.5 秒时限绰绰有余。

递归伪代码

```
dfs(pos, chosen, sum):  
    if chosen == k:  
        if isPrime(sum): ans++  
        return  
    if pos == n: return  
    dfs(pos+1, chosen+1,  
        sum + a[pos])    // 选它  
    dfs(pos+1, chosen,  
        sum)              // 不选
```

一句话记住

为什么不用排列？加法交换律：3+7 与 7+3 是同一个和——组合枚举天然免掉这类重复。

方法拆解 + 样例手推：答案是 1

手推三步

① 列组合

4 选 3 共 4 种：12、19、19、19。

② 判素

22 = 2×11 $\times$ ；29 试除 2、3、5 (5×5 > 29 停) → 素 $\checkmark$ ；34、38 偶数直接 $\times$ 。

③ 计数

恰好 1 种 $\checkmark$ 官方样例。

便宜的剪枝

偶数先排除是最省的筛法：isPrime 开头查 $x < 2$ 、试除从 2 开始，已把一半组合挡在门外。

组合	和	判素过程	结论
{3,7,12}	22	2×11	非素
{3,7,19}	29	试除 2,3,5 无整除	素 $\checkmark$
{3,12,19}	34	偶数	非素
{7,12,19}	38	偶数	非素

手推结论

DFS 的选/不选展开共 $2^4 = 16$ 条路径，比 4 种组合多——多余路径靠 chosen/pos 边界自然剪掉，不用手工去重。

```
#include <iostream>
using namespace std;
int n, k, a[25], ans = 0;
bool isPrime(long long x) {
    if (x < 2) return false;
    for (long long i = 2; i * i <= x; i++)
        if (x % i == 0) return false; // 试除到根号
    return true;
}
void dfs(int pos, int chosen, long long sum) {
    if (chosen == k) {                // 选够 k 个
        if (isPrime(sum)) ans++;
        return;
    }
    if (pos == n) return;            // 没得选了
    dfs(pos + 1, chosen + 1, sum + a[pos]); // 选它
    dfs(pos + 1, chosen, sum);         // 不选
}
int main() {
    cin >> n >> k;
    for (int i = 0; i < n; i++) cin >> a[i];
    dfs(0, 0, 0);
    cout << ans << endl;
    return 0;
}
```

结构一览

全局 n 、 k 、 $a[]$ 、 ans + 两个函数——第一次出现递归：dfs 调用 dfs，像“套娃”一样把选/不选的岔路走完。

三步对号入座

两个 if = 两条边界；两个 dfs = 两条岔路。isPrime 只除到根号 sum，对应第 30 页的复杂度账。

两个好习惯

① sum 用 long long（和可达 10^8 ）；② 试除变量 i 也用 long long， $i * i \leq x$ 才不会溢出。

提交提示

课后亲手敲一遍并提交洛谷 P1036。官方样例 4 3 → 1 已通过验证脚本验证；递归模板第 21 课还会反复出现。

题面描述

猪猪 Hanke 有 10 种配料，每种可以放 1~3 克。烤鸡的美味程度 = 所有配料质量之和。给定美味程度 n，输出所有搭配方案。

输出格式：第一行方案总数；随后每行 10 个数，按字典序排列。无解只输出一个 0。

本课定位：多维全枚举 + 可行性剪枝——"枚举层由约束决定"的活教材。

思路预告

十层 for 也行，但写起来伤手——用递归把「第 i 层循环」变成「第 i 种配料」，顺带见识第一味剪枝。

输入格式

一个正整数 n（美味程度）。

输出格式

方案总数 + 每行 10 个数的方案列表（字典序）；无解输出 0。

数据范围

n ≤ 30。

官方样例

输入	输出
11	10 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 1 ... (共 10 行)

10 种方案 = 全 1 基础上把某一位换成 2——"2"的位置从右往左扫过。

思路梳理：递归枚举 + 剪枝

三个要点

递归枚举 = 十层循环

dfs(i, s): 已定前 i 种、当前和 s。i = 10 时 s == n 就收方案；否则 v = 1..3 枚举第 i+1 种——与十层 for 完全等价。

剪枝：不进死胡同

剩下 rest 种配料至少贡献 rest 克、至多 3 · rest 克。s + v + rest > n 时连更大的 v 也不用试 (break)；够不着下限就 continue。

为什么先存后输出

第一行要先打印方案总数——边搜边把方案存进 plans 数组，搜完统一打印。v 从小到大枚举，输出天然字典序。

递归伪代码

```
dfs(i, s):  
    if i == 10:  
        if s == n: 收方案  
        return  
    for v = 1 .. 3:  
        rest = 9 - i  
        if s+v+rest > n: break  
        if s+v+3·rest < n: continue  
        cur[i] = v  
        dfs(i+1, s+v)
```

剪枝的分量

n = 1 时无剪枝要空跑 $3^{10} = 59049$ 条路径，剪枝后几乎立刻归零——可行性剪枝是枚举的"急刹车"。

方法拆解 + 样例手推：n = 11 → 10 条

手推三步

① 首条路径

前 9 种全放 1（和 9），第 10 种被迫放 2——第一条方案 (1×9, 2)。

② 回溯

第 9 种改放 2 → (⋯, 2, 1)；再退到第 8 种……"2"的位置一路左移，共 10 条。

③ 收工

√ 从小到大 + 回溯，输出顺序与官方样例逐行一致 √。

容量心里有数

方案最多多少？n = 20 时约 8953 种（组合数学可算）——plans 数组开 6 万富余，永远不会溢。

路径	前 9 种	第 10 种	和
第 1 条	全 1	2	11 √
第 2 条	第 9 种 = 2，其余 1	1	11 √
第 3 条	第 8 种 = 2，其余 1	1	11 √
.....	「2」继续左移	——	共 10 条

手推结论

10 条方案与官方样例逐行一致（已用验证脚本核对）。无解时（如 n = 1）一条方案都收不到，输出 0。

参考代码：递归 + 可行性剪枝

```
#include <iostream>
using namespace std;
int n, cur[10], cnt = 0;
int plans[60000][10]; // 方案数组, 留足富余
void dfs(int i, int s) {
    if (i == 10) { // 收口
        if (s == n) { // 收一条方案
            for (int j = 0; j < 10; j++) plans[cnt][j] = cur[j]; cnt++;
        }
        return;
    }
    for (int v = 1; v <= 3; v++) {
        if (s + v + (9 - i) > n) break; // 超了
        if (s + v + 3 * (9 - i) < n) continue;
        cur[i] = v; dfs(i + 1, s + v);
    }
}
int main() {
    cin >> n; dfs(0, 0);
    cout << cnt << "
"; // 方案总数
    for (int i = 0; i < cnt; i++) {
        for (int j = 0; j < 10; j++) cout << plans[i][j] << " "; cout << "
";
    }
}
```

结构一览

cur[10] 记当前方案、plans[][] 存全部方案、cnt 计数——递归函数 dfs 只有十几行，撑起十层循环的全部逻辑。

三步对号入座

if (i == 10) = 收口验货; for (v...) = 每层枚举; 两条 if = 可行性剪枝。与第 34 页伪代码逐行对应。

两个好习惯

① break / continue 用对：超了直接 break（后面更大），不够才 continue；② 方案数组留足富余，防爆数组也防爆栈。

提交提示

课后亲手敲一遍并提交洛谷 P2089。官方样例 11 → 10 行方案（顺序敏感！）已通过验证脚本逐行核对。

题面描述

n 行 m 列的雷区中，* 表示地雷格，? 表示非地雷格。求每个非雷格周围 8 个方向上地雷格的数目，雷格保持 * 不变。

「周围」=上、下、左、右、四个斜角，共 8 个直接相邻的格子。

本课定位：二维网格枚举——"枚举格子 × 枚举方向"的标配套路，也是搜索的地基。

思路预告

每个非雷格都要"看一圈邻居"——8 个方向的偏移量写进数组，一行 for 干完，别手写 8 个 if。

输入格式

第一行 n m；接下来 n 行每行一个长度为 m 的字符串（* 或 ?，无分隔符）。

输出格式

n 行：雷格输出 *，非雷格输出周围雷数。

数据范围

1 ≤ n, m ≤ 100。

官方样例

输入	输出
3 3	*10
*??	221
???	1*1
?*?	

逐格数邻居——(1,1) 周围两个雷，所以是 2。

三个要点

方向数组

$dx = 1, dy = 1$ ——8 个相对位移——定义好，d 从 0 到 7 一轮循环看完所有邻居。

边界判断

邻居坐标 $x = i + dx[d]$ 、 $y = j + dy[d]$ ，必须 $0 \leq x < n$ 且 $0 \leq y < m$ 才算数——出界的"邻居"不存在。

复杂度

$n \cdot m \cdot 8 = 8$ 万次判断，1 秒时限闭眼过——枚举格子 $\times$ 枚举方向，双重循环的标配套路。

主循环伪代码

```
读入 n, m 和 g[][]  
  
for i = 0 .. n-1:  
    for j = 0 .. m-1:  
        if g[i][j] == '*': 原样输出  
    else:  
        c = 0  
        for d = 0 .. 7:  
            x = i+dx[d]; y = j+dy[d]  
            if 合法 且 g[x][y]=='*':  
                c++  
        输出 c
```

向前看

方向数组是搜索、BFS、DP 转移的公共地基——今天写熟，第 23 课 BFS 直接受益。

方法拆解 + 样例手推：拼出 *10 / 221 / 1*1

手推三步

① 看 (0,1)

8 邻居里只有 (0,0) 是 * $\rightarrow$ 数字 1。

② 看 (1,1)

邻居 (0,0)、(2,1) 都是 * $\rightarrow$ 数字 2。

③ 拼全图

逐格扫完 $\rightarrow$ *10 / 221 / 1*1 $\checkmark$ 官方样例。

画个九宫格

别被"斜角"骗了：dx、dy 里 ± 1 的组合恰好覆盖 8 个方向——画个 3×3 九宫格一眼看清。

格子	雷邻居	输出
(0,1)	(0,0)	1
(0,2)	无	0
(1,0)	(0,0), (2,1)	2
(1,1)	(0,0), (2,1)	2
(1,2)	(2,1)	1

手推结论

手推表与官方样例输出 *10 / 221 / 1*1 完全一致——第二组样例 2 3 $\rightarrow$ 2*1 / *21 课后自验。

```
#include <iostream>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    char g[105][105];
    for (int i = 0; i < n; i++) cin >> g[i];
    int dx[8] = {-1,-1,-1,0,0,1,1,1};
    int dy[8] = {-1,0,1,-1,1,-1,0,1};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (g[i][j] == '*') { cout << '*'; continue; }
            int c = 0; // 数雷邻居
            for (int d = 0; d < 8; d++) {
                int x = i + dx[d], y = j + dy[d];
                if (x >= 0 && x < n && y >= 0 && y < m
                    && g[x][y] == '*') c++;
            }
            cout << c;
        }
        cout << "
";
    }
    return 0;
}
```

结构一览

g[105][105] 存雷区、dx/dy 两个方向数组、双重循环 + 内层 8 向循环——四层嵌套读起来吓人，其实每层职责单一。

三步对号入座

外两层 = 枚举格子；内层 = 枚举方向；if (合法 且 是雷) = 检验 + 计数。与第 38 页伪代码——对应。

两个好习惯

- ① 数组开 105 留边界余量；② 雷格用 continue 跳过，逻辑分流更清楚。

提交提示

课后亲手敲一遍并提交洛谷 P2670。两组官方样例均已通过验证脚本验证。

T09 | 统计天数（P1567）

题面描述

KC 收集了连续 n 天的最高气温数据，想知道气温「一直上升」的最长连续天数。

「一直上升」=严格递增：每天的气温必须比前一天高，相等也算断。

本课定位：线性扫描——枚举家族里最朴素也最常用的一支，1 遍跑完。

思路预告

只需要"扫一遍"：记住当前上升段多长、最长段多长，两个变量走完全程。

输入格式

第一行一个整数 n ；第二行 n 个整数，表示连续 n 天的最高气温。

输出格式

一行一个整数：最长连续上升天数。

数据范围

$1 \leq n \leq 10^6$ （百万级数据，读入要加速）。

官方样例

输入	输出
10 1 2 3 2 4 5 6 8 5 9	5

最长的一段是 2 4 5 6 8——从第 4 天到第 8 天。

思路梳理：一遍扫描两个变量

三个要点

为什么一遍就够

"上升"是相邻天之间的关系：每天只和昨天比。比较结果只有两种——继续升 / 断了，信息不丢，不用回头看。

两个变量

cur: 当前上升段长度（含今天）；best: 历史最长。 $a[i] > a[i-1]$ 时 cur++ 并刷新 best；否则 cur = 1 从今天重数。

初值与细节

best 初值是 1 ($n \geq 1$ 时至少 1 天)；相等不算上升（严格大于）；n 达 10^6 要关同步流加速读入。

主循环伪代码

```
读入 n 和 a[0..n-1]

cur = 1;  best = 1

for i = 1 .. n-1:
    if a[i] > a[i-1]:
        cur++
        best = max(best, cur)
    else:
        cur = 1

输出 best
```

另一层收获

这道题没有第二层循环可优化——它教的是：枚举对象选「相邻关系」而不是「所有区间」， $O(n^2)$ 直接降为 $O(n)$ 。

方法拆解 + 样例手推：答案是 5

手推三步

① 分段

1 2 3 | 2 4 5 6 8 | 5 9——三段上升段，长度 3、5、2。

② 跟踪

段 1: cur 1→2→3; 段 2: cur 1→2→3→4→5, best 刷到 5★; 段 3: cur 1→2。

③ 输出

best = 5 ✓ 官方样例。

数段口径

第一个 2 既是上一段的结尾又是下一段的开头——数段时以"起点"重计，别数重。

段	天气序列	cur 最终	best
第 1 段	1 2 3	3	3
第 2 段	2 4 5 6 8	5	5 ★
第 3 段	5 9	2	5

手推结论

best = 5，与官方样例一致。这个"分段扫描"套路在第 9 课递推（最大连续和）会升级回归。

```
#include <iostream>
#include <algorithm>
using namespace std;
int a[1000005];
int main() {
    ios::sync_with_stdio(false); // 百万级加速
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    int cur = 1, best = 1; // 至少 1 天
    for (int i = 1; i < n; i++) {
        if (a[i] > a[i - 1]) {
            cur++; // 还在上升
            best = max(best, cur);
        } else cur = 1; // 断了重数
    }
    cout << best << endl;
    return 0;
}
```

结构一览

a[1000005] 一个数组、cur / best 两个变量、一层循环——全课最短的参考代码，重点全在"初值"与"严格大于"。

三步对号入座

cur++ + max = 上升分支；cur = 1 = 断裂分支。与第 42 页伪代码逐行对应。

两个好习惯

① ios::sync_with_stdio(false) + cin.tie(0) 应对百万级读入；② best 初值 1，别忘了 n = 1 的边界。

提交提示

课后亲手敲一遍并提交洛谷 P1567。官方样例 10 天 → 5 已通过验证脚本验证。

T10 | 欢乐的跳（P1152）

题面描述

一个 n 个元素的整数数组，如果相邻两数之差的绝对值恰好包含 $1 \sim n-1$ 的所有整数，就称它「欢乐的跳」。判断给定数组是否符合。

例如：1 4 2 3：差的绝对值 3、2、1，恰好齐了 → Jolly。

本课定位：桶标记判"齐全"——检验环节的收官题。

思路预告

差值只有 $n-1$ 个，目标范围也是 $1 \sim n-1$ ——开一排"小格子"打勾，"齐全"两个字就有得查。

输入格式

每组数据第一行一个整数 n ，随后 n 个整数（数值较大）。多组数据，读到文件尾。

输出格式

每组一行：符合输出 Jolly，否则输出 Not jolly。

数据范围

$3 \leq n \leq 10^4$ ；元素绝对值可达 10^8 （long long 稳妥）。

官方样例

输入	输出
4 1 4 2 3	Jolly
5 1 4 2 -1 6	Not jolly

两组样例一勾一漏，恰好覆盖两个输出分支。

三个要点

枚举差值

相邻对 (a[i-1], a[i]) 逐对枚举, $d = |a[i] - a[i-1]|$ ——又是"枚举对象 = 相邻关系"的一例。

桶标记

seen[d] = true 给差值 d 打勾。只有 $1 \leq d \leq n-1$ 才入桶——越界的差说明"名额被挤占", 检验时自然失败。

检验与多组数据

最后查 $1 \sim n-1$ 是否全打过勾; while (还能读入 n) 读到文件尾, 每组记得把 seen[] 清零。

主循环伪代码

```
while (还能读入 n):  
    读入 a[0..n-1]  
    seen[] 清零  
    for i = 1 .. n-1:  
        d = |a[i] - a[i-1]|  
        if 1 ≤ d ≤ n-1:  
            seen[d] = true  
    ok = 1..n-1 全部 seen  
    输出 Jolly / Not jolly
```

逻辑自动兜底

为什么差值 0 不用特判? 需要的差从 1 开始, 0 出现只会挤掉别的差 → 检验必失败, 无需额外分支。

手推三步

① 样例 1

1 4 2 3: 差 3、2、1 → 勾住 1、2、3, 1..3 齐全 → Jolly。

② 样例 2

1 4 2 -1 6: 差 3、2、3、7 → 勾住 2、3 (7 越界不入桶) → 1..4 缺 1 → Not jolly。

③ 负数也别怕

$a[i] - a[i-1]$ 可能是负的——取绝对值再入桶，llabs 一行搞定。

查覆盖不查重复

差值重复出现（样例 2 的两个 3）不是错——错的是"该到的没到"。检查的是覆盖，不是重复。

数组	差序列	入桶的勾	结论
1 4 2 3	3 2 1	1, 2, 3	Jolly
1 4 2 -1 6	3 2 3 7	2, 3	Not jolly

手推结论

两行样例一勾一漏，恰好覆盖 Jolly / Not jolly 两个分支——官方样例的选择是有讲究的。

```
#include <iostream>
#include <cmath>
using namespace std;
long long a[10005];
bool seen[10005];      // seen[d]: 差值d出现过吗
int main() {
    int n;
    while (cin >> n) {    // 多组数据到文件尾
        for (int i = 0; i < n; i++) cin >> a[i];
        for (int d = 0; d <= n; d++) seen[d] = false;
        for (int i = 1; i < n; i++) {
            long long d = labs(a[i] - a[i - 1]);
            if (d >= 1 && d <= n - 1) seen[d] = true;
        }
        bool ok = true;
        for (int d = 1; d <= n - 1; d++)
            if (!seen[d]) ok = false;    // 缺哪个都不行
        cout << (ok ? "Jolly" : "Not jolly") << "
";
    }
    return 0;
}
```

结构一览

seen[10005] 一排小勾、while 多组读入、一层循环算差——结构比 T07 简单，难点全在“想清楚检验什么”。

三步对号入座

for = 枚举差值；seen[d] = 打勾；最后的 for = 验齐全。与第 46 页伪代码一一对应。

两个好习惯

① a[] 与差值用 long long (10⁸ 级别)；② seen[] 每组数据重置——多组输入的老坑，错一次记一生。

提交提示

课后亲手敲一遍并提交洛谷 P1152。两组官方样例均已通过验证脚本验证。

① 范围开错，白白漏解

三连击只枚举 $123 \sim 329$ ($z = 3x \leq 981$ 是三位数)；烤鸡每位 $1 \sim 3$ ，不是 $0 \sim 3$ 。范围由「数据范围 + 隐含条件」共同推出，写循环前先在纸上把上下界算死。

② 多组数据，数组忘清零

欢乐的跳、A-B 数对这类题常见多组输入——桶和计数数组必须在每组开头清零。判题机上第一组对、第二组错，九成是这个原因。

③ 边界差一，答案偏一格

统计天数上升段计数从 1 起步（单天也是一段）；爱与愁的心痛窗口长度固定为 m ，滑到 $n - m$ 为止。写完先用样例手推一遍下标走向。

④ 范围爆 int，结果全错

优秀的拆分 $n \leq 10^7$ 、A-B 数对数值达 10^9 ——该 long long 的别省。判断依据：最大值 $\times$ 运算次数是否接近 $2^{31} \approx 2.1 \times 10^9$ 。

⑤ 输出格式，隐性丢分

烤鸡无解要输出 0 而不是空行；扫雷雷格保留 *；优秀的拆分 -1 单独输出。OJ 对格式零容忍，样例怎么给就怎么输出。

课堂总结：枚举的「三步 + 三刀」

枚举三步法（拿到题先问自己）

- ① 枚举什么？——把「解」的形态定下来：一个数、一对数、一个方案、一张网格。
- ② 按什么顺序枚举？——不重不漏，字典序/从大到小/逐格扫描。
- ③ 怎么检验？——检验逻辑要 $O(1)$ 化：桶标记、方向数组、预计算。

给枚举砍三刀（本课的「优化」）

- 第一刀 · 砍范围：数学性质缩小候选集（奇数无解 → 一半数据免算）。
- 第二刀 · 砍层数：利用对称/独立性质少一层循环（A-B 数对改查桶）。
- 第三刀 · 砍检验：预处理 + 剪枝，让每次检验从 $O(n)$ 降到 $O(1)$ 。

T01 优秀的拆分	数学性质	二进制 → 24 轮贪心	性质砍范围
T02 三连击	缩小枚举对象	只枚举 x: 123~329	对称砍层数
T03 爱与愁的心痛	滑动窗口	$O(n \cdot m) \rightarrow O(n)$	预处理砍检验
T04 A-B 数对	桶 / map 计数	$O(n^2) \rightarrow O(n)$	查表砍检验
T05 火柴棒等式	预计算 + 对称	枚举 a,b 查 a+b	预处理砍检验
T06 选数	组合 + 试除	$C(n,k)$ 枚举判素	检验要快
T07 烤鸡	DFS + 剪枝	可行性剪枝砍分支	剪枝砍范围
T08 扫雷	方向数组	格子 × 8 向	套路化检验
T09 统计天数	线性扫描	一次遍历计数	最朴素也最快

枚举：暴力，但是聪明的暴力

把所有可能摆上桌面，再用数学性质砍到只剩必要的那部分

课后任务：完成 T02 ~ T10 提交 · 重做 T01 变式 ($n \leq 10^9$ 时还能更快吗？)

第 4 课预告 · 排序与去重